MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 3

Operating Systems

Topic: The Command Line: Shell, Pipes and Text Filters

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Purpose of the Work 3

2 Initial Setup 3

3 Part 1 – Move Fast, Find Help 3

3.1 Observations . . . 3

4 Part 2 – What the Shell Does to Your Line 5

4.1 Observations . . . 6

5 Part 3 – Redirection, Chaining and Jobs 7

5.1 Observations . . . 7

6 Part 4 – Answer Questions with One-Liners 8

6.1 Q1 – How many requests failed? . . . 8

6.2 Q2 – How many different pages were requested, and which? . . . 8

6.3 Q3 – How many requests did each page get? . . . 8

6.4 Q4 – Which user or users have the most failed requests, and how many? . . . 9

6.5 Q5 – How many requests did user3 make, and how many of them failed? . . . 9

6.6 Q6 – Print the last 3 failed requests, showing only time and user . . . 9

6.7 Q7 – Which login shells appear in /etc/passwd, and how many accounts use each? . . . 9

6.8 Q8 – Why do the two line counts differ by exactly one? . . . 9

6.9 The oneliners.sh File . . . 9

7 Part 5 – My First Script 10

7.1 Observations . . . 11

8 Conclusion 12

1. Purpose of the Work

The purpose of this laboratory work is to become faster in Bash, understand what the shell does to a command line before it runs, redirect output where it is needed, chain commands, work with background jobs, and answer questions about text data using small filters connected with pipes.

2. Initial Setup

The laboratory uses an artificial web access log with 500 entries. I worked inside `~/os-lab3` and used the provided data set throughout the exercises.

Dataset setup

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5))
    -eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
head -3 access.log ; wc -l access.log
```

The generated `access.log` contains 500 lines, which is the expected size for the rest of the laboratory.

3. Part 1 – Move Fast, Find Help

I first checked which commands are shell built-ins and which are external programs, where the shell searches for programs, what happens when a command does not exist, and how to use local manual pages and help.

Commands

```
type cd ; type ls ; type echo ; which ls
echo $PATH
echo $PATH | tr ':' '\n'
nosuchprog ; echo "exit code: $?"
ls --help | head
man -k "list directory"
help cd
```

3.1. Observations

1. Of `cd`, `ls` and `echo`, only `echo` and `cd` are built in. `cd` must be built in because when a shell calls an external command, it creates a child process which runs the command. The child could successfully change its directory, but when it ends, control returns to the shell and that state would not be saved for the shell itself.



Figure 1.1 – Output of `type`, `which`, the directories in `PATH`, and the exit code of a missing command.

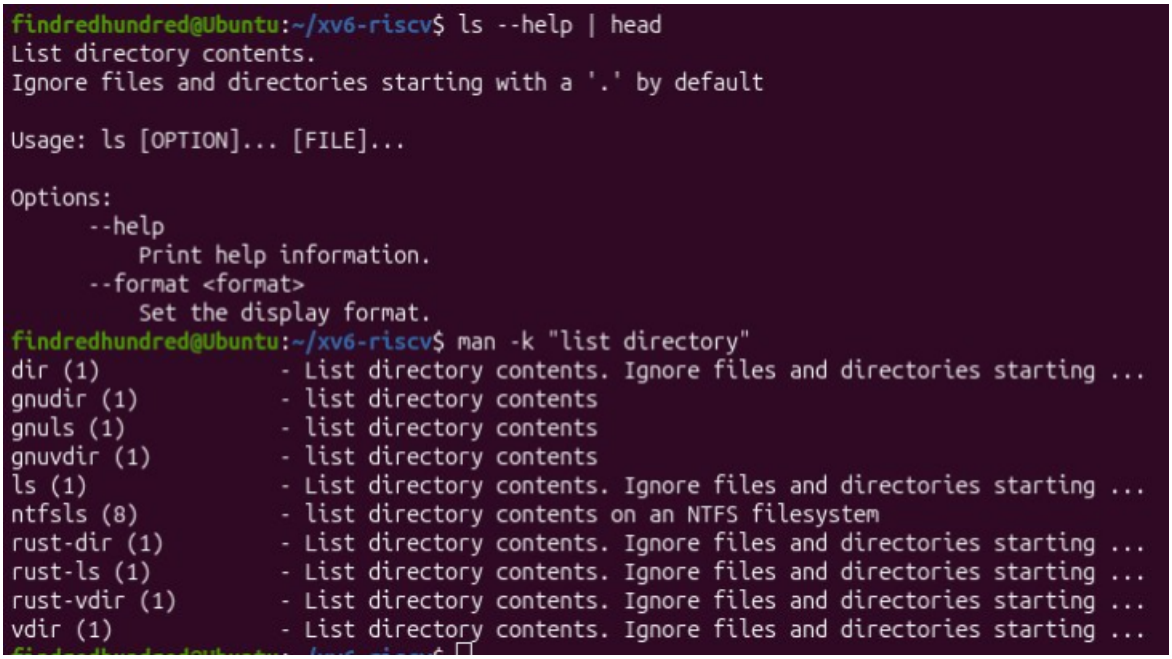


Figure 1.2 – Finding help with `ls -help` and `man -k`.

2. There are 10 directories in my `PATH`. The path of `ls` is `/usr/bin`; this can also be seen by writing `whereis ls`.
3. The exit code for a command that is not found is `127`.
4. `ls -t` sorts by modification time. The help describes it as sorting by modification time, newest first.

4. Part 2 – What the Shell Does to Your Line

I created a small test directory containing ordinary file names and one file name containing a space.

Setup

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

Before running the commands, I wrote down my predictions and then compared them with the real output.

Command	My prediction	Real output	Explanation if different
`echo *.c`	`a.c b.c`	`a.c b.c`	Same as predicted.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	Same as predicted.
`echo [a-b]*`	`a.c alfa beta.txt b.c`	`a.c alfa beta.txt b.c`	Same as predicted.
`echo *.xyz`	`.xyz`	`*.xyz`	There was no matching file, so the unmatched glob stayed literal.
`ls alfa beta.txt`	cannot find `alfa`	two “No such file or directory” errors	The shell treated the space as an argument separator.
`ls "alfa beta.txt"`	`alfa beta.txt`	`'alfa beta.txt'`	Same file; `ls` quotes the displayed name to make the space clear.
`echo '$HOME' "$HOME" \$HOME`	`$HOME /home/findredhundred $HOME`	`$HOME /home/findredhundred $HOME`	Same as predicted.
`echo {1..5}; echo $((7*6))`	`1 2 3 4 5` then `42`	`1 2 3 4 5` then `42`	Same as predicted.
`echo "today is $(date +%A), I am $(whoami)"`	Sunday, `findredhundred`	Monday, `findredhundred`	The weekday came from the date at the moment the command ran.

Command	My prediction	Real output	Explanation if different
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	`log1.txt log2.txt log3.txt`	Same as predicted.

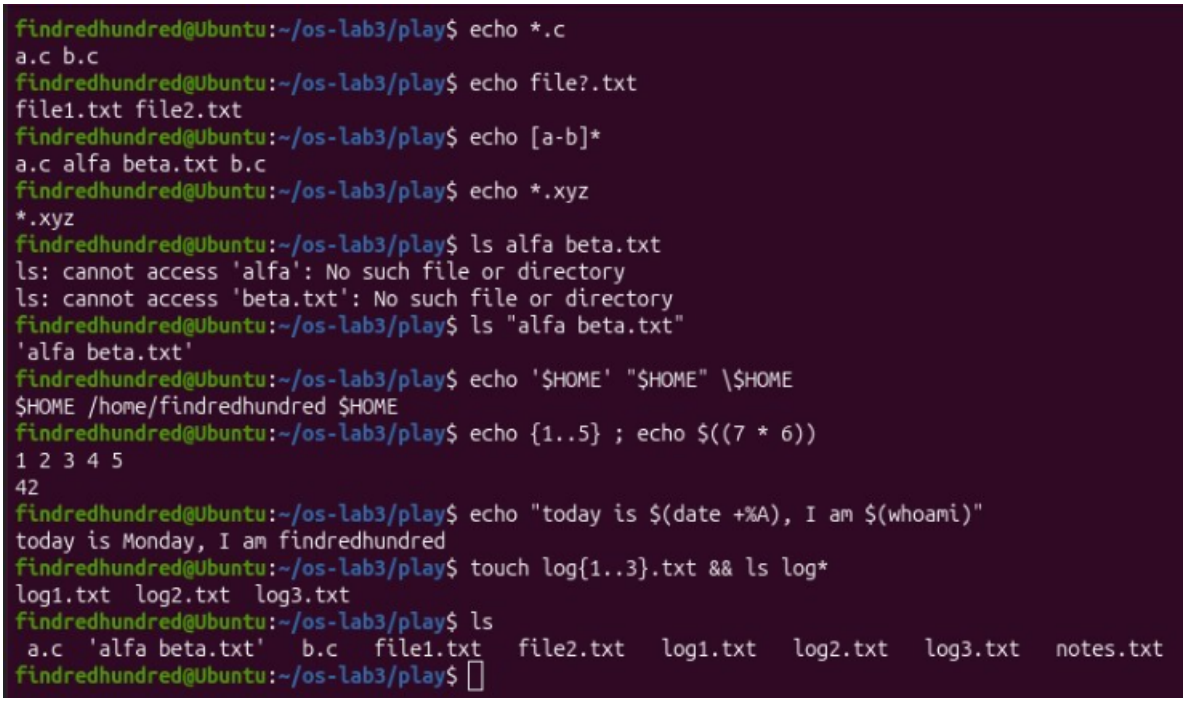


Figure 2.1 – Globbing, quoting, command substitution, brace expansion, and file names containing spaces.

4.1. Observations

Most of my predictions were on point. The main visible differences were the unmatched `*.xyz` pattern and the weekday printed by the command substitution.

1. `echo` itself knows nothing about files. The shell produced the file names. The glob is expanded by the shell before it is passed to the program, so the shell replaced the pattern with the matching files in my directory and then passed them to `echo`.

2. `ls alfa beta.txt` failed because the space between `alfa` and `beta.txt` makes the shell think they are two separate file names. Two ways to fix it are `ls "alfa beta.txt"` or escaping the space: `ls alfa\ beta.txt`.

3. The difference between single and double quotes is that single quotes treat everything inside as literal characters, while double quotes can still expand variables and command substitutions.

4. `ls "alfa beta.txt"` displays the file as `'alfa beta.txt'` to make it clear that the whole thing is one file name and not two separate outputs.

5. Part 3 – Redirection, Chaining and Jobs

I tested separate redirection of standard output and standard error, combined redirection, appending to a file, input redirection, conditional chaining, and background jobs.

Commands and output

```
cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt ; cat err.txt
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
echo first > notes.md ; echo second >> notes.md ; cat notes.md
wc -l < access.log
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
sleep 100 &
jobs ; kill %1 ; jobs
```

```
findredhundred@Ubuntu:~/os-lab3/play$ cd ~/os-lab3
findredhundred@Ubuntu:~/os-lab3$ ls /etc /nope > out.txt 2> err.txt
findredhundred@Ubuntu:~/os-lab3$ wc -l out.txt ; cat err.txt
241 out.txt
ls: cannot access '/nope': No such file or directory
findredhundred@Ubuntu:~/os-lab3$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xml
zsh_command_not_found
findredhundred@Ubuntu:~/os-lab3$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
findredhundred@Ubuntu:~/os-lab3$ wc -l < access.log
500
findredhundred@Ubuntu:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
findredhundred@Ubuntu:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/findredhundred/os-lab3/tmp
findredhundred@Ubuntu:~/os-lab3$ sleep 100 &
[1] 15761
findredhundred@Ubuntu:~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
findredhundred@Ubuntu:~/os-lab3$
```

Figure 3.1 – Redirection, appending, input redirection, command chaining, and a background job.

I also compared the order of two redirections:

Redirection order

```
ls /etc /nope > all.txt 2>&1
ls /etc /nope 2>&1 > all.txt
```

5.1. Observations

1. In `ls /etc /nope > out.txt 2> err.txt`, `out.txt` got standard output and `err.txt` got standard error.

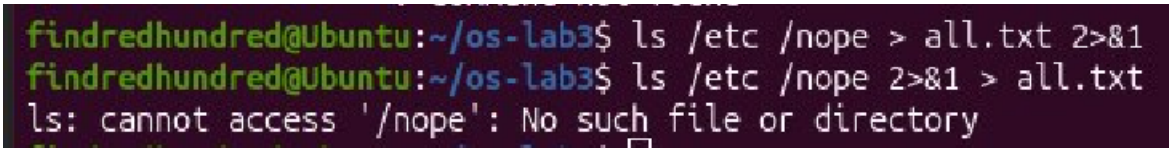


Figure 3.2 – Comparing the order of standard-output and standard-error redirection.

2. The difference between `> all.txt 2>&1` and `2>&1 > all.txt` is the order in which the redirections are applied. The first sends both stdout and stderr to `all.txt`; in the second, stderr is first connected to the current stdout, and only after that stdout is redirected to `all.txt`.

3. The difference with `wc -l < access.log` is in who opens the file. When `wc` opens the file itself, it also knows the file name and prints it next to the count. With `<`, the shell opens the file and feeds its contents to `wc`, so `wc` never sees the file name and prints only the line count.

4. `;` runs the second command no matter what. `&&` runs the second command only if the first returned successfully. `||` runs the second command only if the first failed.

6. Part 4 – Answer Questions with One-Liners

The access log has the format: date, time, user, result, page. I answered each question using one command line and saved the commands in `oneliners.sh`.

6.1. Q1 – How many requests failed?

```
grep -i "FAIL" access.log | wc -l
```

The result is 100 failed requests.

6.2. Q2 – How many different pages were requested, and which?

```
cut -d'/' -f2 access.log | sort -u | awk '{print; count++} END {print "\nTotal
    pages:", count}'
```

6.3. Q3 – How many requests did each page get?

```
cut -d'/' -f2 access.log | sort | uniq -c
```

6.4. Q4 – Which user or users have the most failed requests, and how many?

```
grep -i 'fail' access.log | cut -d' ' -f3 | sort | uniq -c | sort -r
```

The run of `report.sh` later confirms that `user3` and `user5` have the highest number, with 15 failures each.

6.5. Q5 – How many requests did user3 make, and how many of them failed?

```
data=$(grep "user3" access.log) ; echo "Nr commands: $(echo "$data" | wc -l)"
    ; echo "failed: $(echo "$data" | grep -c "FAIL")"
```

6.6. Q6 – Print the last 3 failed requests, showing only time and user

```
grep "FAIL" access.log | tail -n 3 | cut -d' ' -f2,3
```

6.7. Q7 – Which login shells appear in /etc/passwd, and how many accounts use each?

```
cut -d':' -f7 /etc/passwd | sort | uniq -c
```

6.8. Q8 – Why do the two line counts differ by exactly one?

When `ls` runs normally inside a pipeline, it outputs one clean file name per line. When the long listing flag is added, `ls -l` prints an extra summary line at the top before the file list. That extra line adds exactly one to the final `wc -l` count.

6.9. The oneliners.sh File

oneliners.sh

```
# Q1. How many requests failed?
grep -i "FAIL" access.log | wc -l

# Q2. How many different pages were requested, and which?
cut -d'/' -f2 access.log | sort -u | awk '{print; count++} END {print "\nTotal
    pages:", count}'

# Q3. How many requests did each page get?
cut -d'/' -f2 access.log | sort | uniq -c
```

```
# Q4. Which user or users have the most failed requests, and how many?
grep -i 'fail' access.log | cut -d' ' -f3 | sort | uniq -c | sort -r

# Q5. How many requests did user3 make, and how many of them failed?
data=$(grep "user3" access.log) ; echo "Nr commands: $(echo "$data" | wc -l)"
    ; echo "failed: $(echo "$data" | grep -c "FAIL")"

# Q6. Print the last 3 failed requests, showing only time and user.
grep "FAIL" access.log | tail -n 3 | cut -d' ' -f2,3

# Q7. Which login shells appear in /etc/passwd, and how many accounts use each
    ?
cut -d':' -f7 /etc/passwd | sort | uniq -c
```

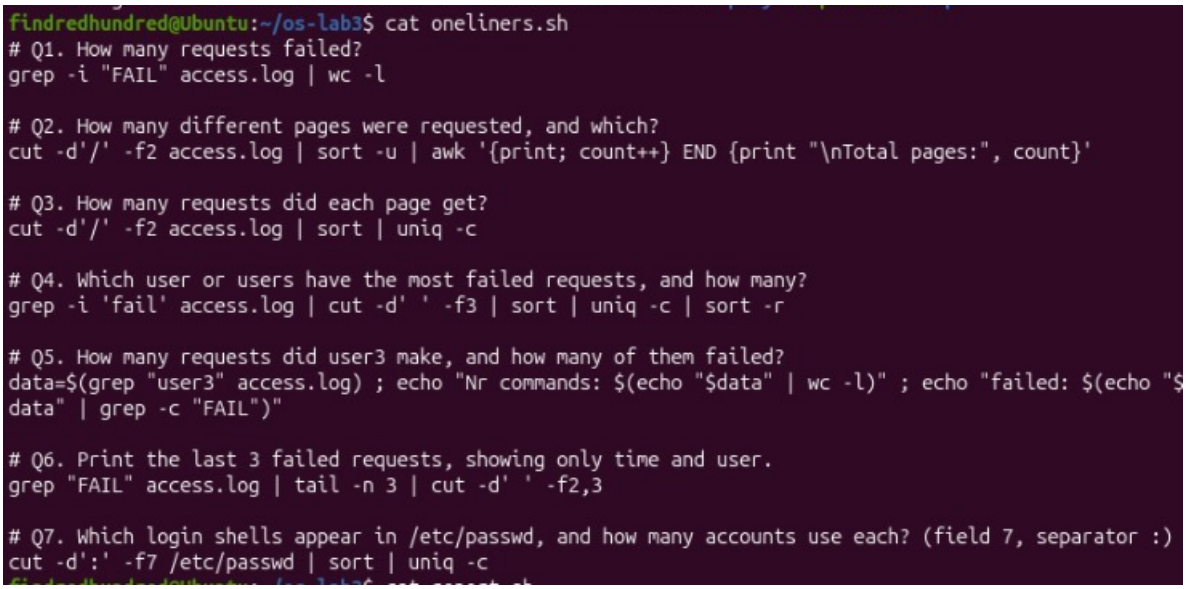


Figure 4.1 – Contents of `oneliners.sh`.

7. Part 5 – My First Script

I turned the one-liners into a small Bash program named `report.sh`. The script checks that the requested log file exists, counts all requests and failed requests, and then loops through users `user0` to `user6` to count failures for each one.

report.sh

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
```

```
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"

# TODO: for each of user0 ... user6, print "userN: X failures"
for i in {0..6}; do
    user_fails=$(grep "user$i" "$log" | grep -c "FAIL")
    echo "user$i: $user_fails failures"
done
```

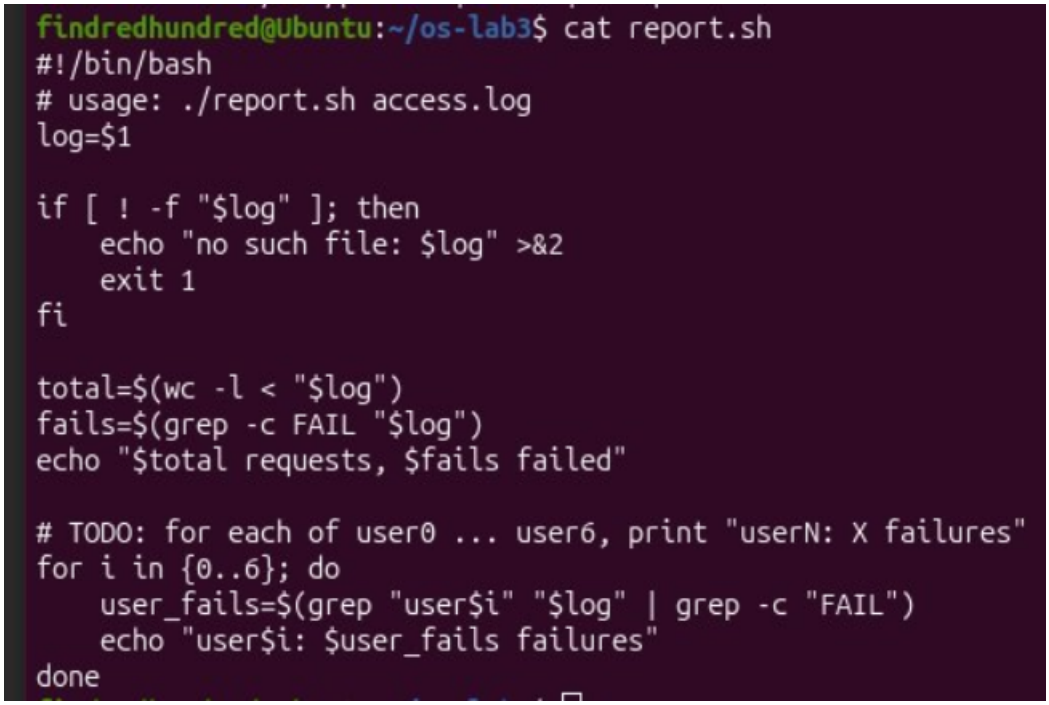


Figure 5.1 – Contents of `report.sh` with the completed loop.

After making the script executable, I ran it on `access.log`. It reported 500 requests and 100 failed requests, followed by the failure count for each user.

7.1. Observations

1. The first line, `#!/bin/bash`, is the shebang. It tells the operating system which interpreter should parse and execute the script, so even if I am using another shell, the script runs with Bash. Before `chmod +x`, running `./report.sh` fails with a permission error because the file does not yet have execute permission.

2. I must type `./report.sh` instead of only `report.sh` because Linux looks for commands in the directories listed in the `PATH` environment variable. The current directory is intentionally not normally part of that path, so `./` explicitly tells the shell to use the file in the current directory.

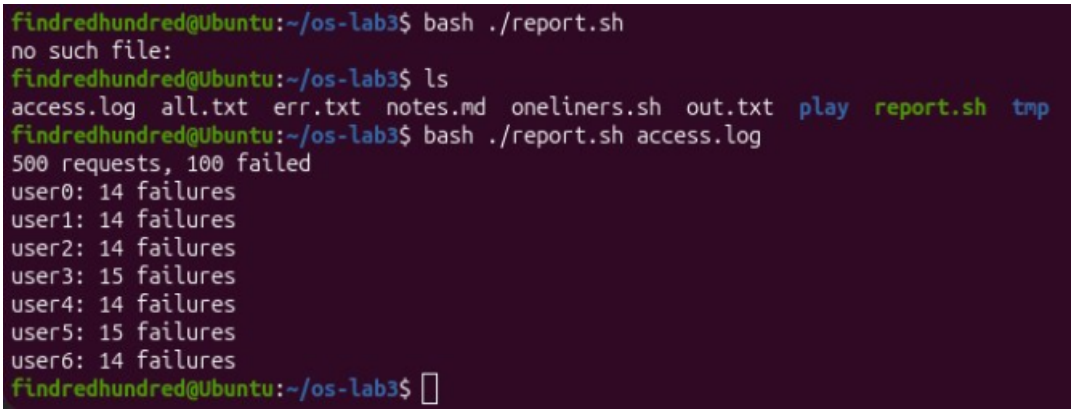


Figure 5.2 – Running `report.sh` and displaying the per-user failure counts.

3. `>&2` sends the error message to standard error instead of normal output. This means that if the script's normal output is redirected, the error can still remain separate. `exit 1` terminates the script with a non-zero exit code, which signals failure and allows other commands or scripts to detect that something went wrong.

8. Conclusion

This laboratory helped me understand what Bash does before a command is actually executed and how much work is done by the shell itself. I practiced finding commands and help, using globbing and quoting, redirecting standard output and standard error, chaining commands with `;`, `&&` and `||`, and controlling background jobs. I also used small text-processing tools together in pipelines to answer questions from `access.log`, then reused the same ideas inside `oneliners.sh` and `report.sh`. The final script made the connection between individual command-line tools and a reusable Bash program much clearer.